



Nitro NLP Hackathon 5th Edition Submission Writeup

Eye-Tracking-Based Reading Time Prediction (👁️📄⏱️)

Team DreamTeam

Răzvan-Matei Dedu Tudor Morariu Mihnea-Teodor Stoica David Ștefan Tache

Summary

This is our submission writeup for the Nitro NLP Hackathon 5th Edition. Nitro describes the hackathon as a 23-hour NLP competition for high-school and bachelor students, organized by the Nitro Association, where teams receive a fresh NLP dataset and build leaderboard-scored models under a short deadline [8].

The task, *Eye-Tracking-Based Reading Time Prediction* (👁️📄⏱️), was to predict Romanian word-level Total Reading Time (TRT), measured from eye-tracking data. Our final solution is a compact tabular model with three main ingredients: word-level target aggregation, rich hand-built lexical and positional features, and frozen Romanian BERT contextual embeddings compressed with PCA. A LightGBM regressor trained on square-root transformed TRT produced a local mean word-level CV score of **87.19** and a row-level out-of-fold score of **44.46**.

1 Task and Data

Each training row contains a word occurrence, participant id, text id, and observed TRT in milliseconds. The test set contains the same word/text identifiers and a `datapointID`, but no target. The official submission is a CSV with `subtaskID`, `datapointID`, and `answer`.

Because train and test `word_ids` do not overlap, we model general word difficulty rather than memorizing word ids. We aggregate training labels by `word_id` using the mean TRT, then map word-level out-of-fold predictions back to the original participant rows for local validation.

Table 1: Dataset summary used by the final solution.

Quantity	Value
Training rows	135,210
Test rows	9,425
Training word ids	4,507
Test word ids	1,885
Train/test word-id overlap	0
Training participants	30
Test participants	5
Distinct training texts	9
Row-level zero-rate	30.62%
Word-level zero-rate after averaging	0.18%
Mean row-level TRT	264.44 ms
Median row-level TRT	200.00 ms

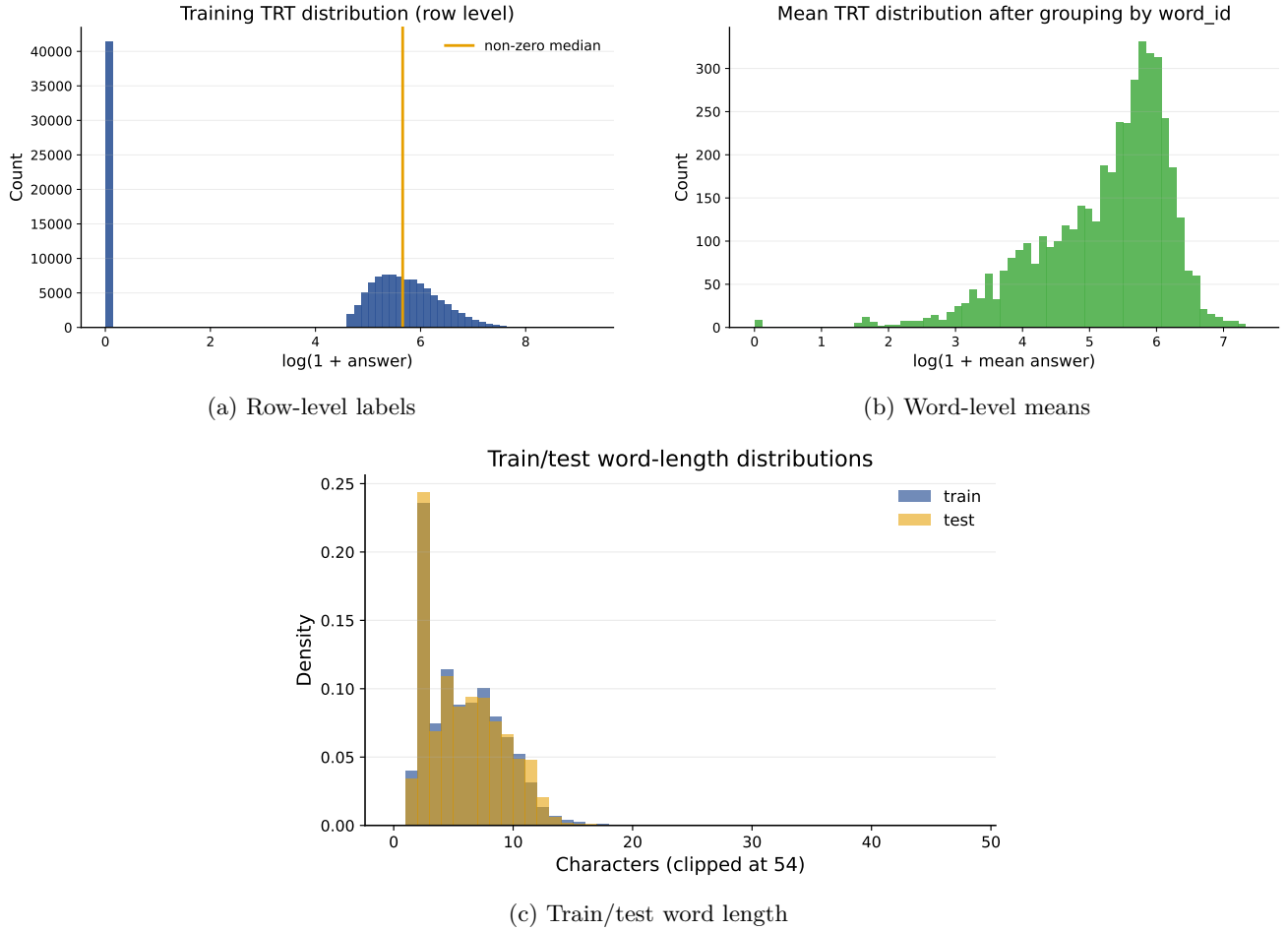


Figure 1: Basic data checks. Raw row-level TRT has many zeros; grouping by `word_id` gives a smoother target.

2 Evaluation Metric

The task metric averages non-negative R^2 and absolute Pearson correlation:

$$S(y, \hat{y}) = 100 \cdot \frac{\max(0, R^2(y, \hat{y})) + |\rho(y, \hat{y})|}{2}. \quad (1)$$

This metric rewards both calibration and ranking. A model can benefit from getting the relative difficulty of words right, but very large magnitude errors still hurt through the R^2 term.

3 Final Pipeline

Our final system is deliberately simple to run once embeddings are cached:

1. Read `train_data.csv`, `test_data.csv`, and `sample_output.csv`.
2. Aggregate training labels by `word_id`.
3. Merge auxiliary Romanian eye-tracking and property CSVs from the public eye-tracking repository [1].
4. Build lexical, position, neighborhood, and categorical word-shape features.
5. Reconstruct sentence-like contexts and embed each target word with Romanian BERT [2, 3].
6. Reduce contextual embeddings to 128 PCA components [5, 10].

7. Train repeated stratified LightGBM folds [6].
8. Average fold predictions and write `output.csv`.

The implementation is provided in both `main.ipynb` and `main.py`. The notebook is a flat procedural version of the solution, matching the way we iterated during the hackathon.

4 Features

Lexical and shape features. We use length, capitalization, punctuation, vowel/consonant counts, syllables, frequency, affixes, and word-shape variants.

Position features. The `word_id` encodes page and within-page position. We derive page index, absolute position, relative page position, distance to page boundaries, and first/last indicators.

Local context features. For core scalar variables we add previous/next one-word and two-word values plus centered three-word rolling means. These features approximate short-range lexical context without needing a sequence model at training time.

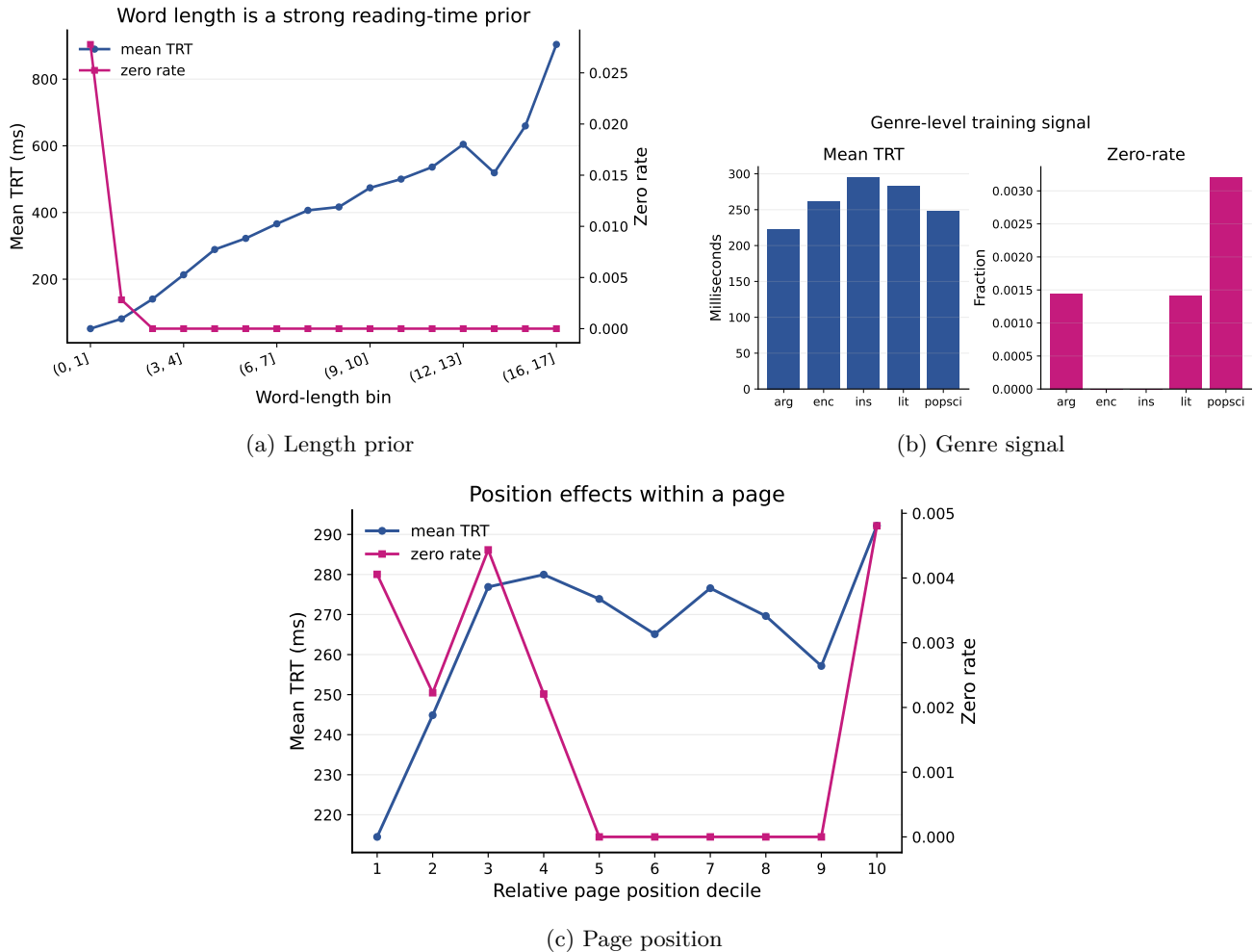


Figure 2: Hand-built features capture meaningful structure before adding neural embeddings.

Auxiliary eye-tracking features. We use Romanian fixation dictionaries and property files from the auxiliary eye-tracking data [1, 4, 7]. The most useful fields are external TRT summaries, positive-fixation rates, sentence-position information, surprisal, token count, and frequency. For numeric auxiliary columns we add raw, log, square-root, and missingness versions.

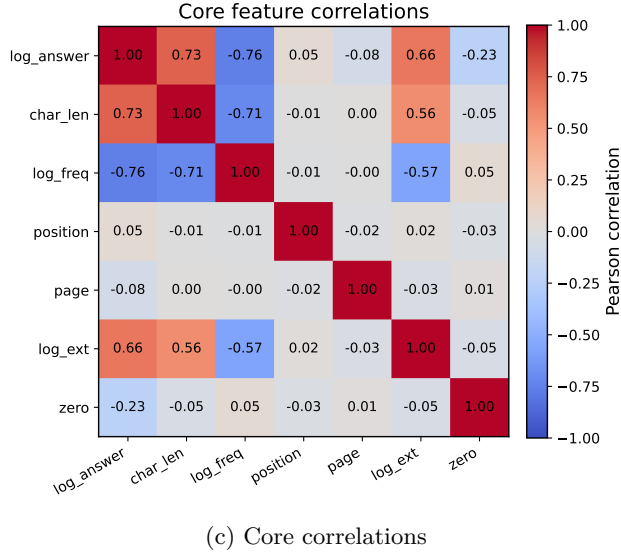
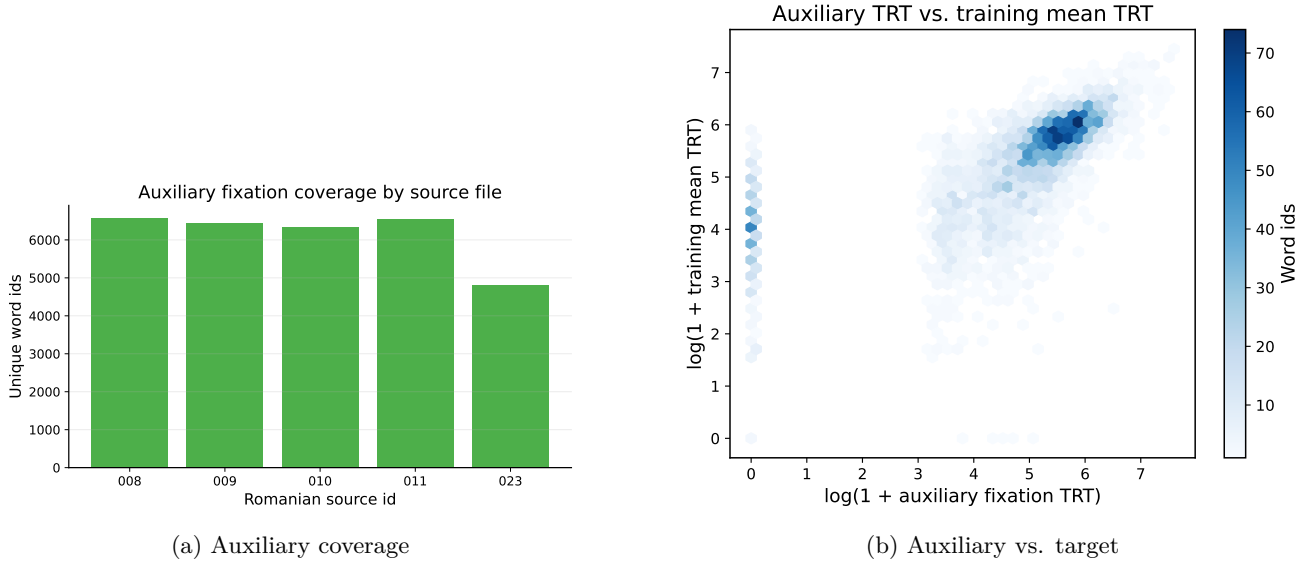


Figure 3: Auxiliary fixation measurements were one of the strongest external signals. The log auxiliary TRT to log training mean TRT correlation is 0.656.

Contextual embeddings. We sort words by text, section, page, and position, then split contexts at sentence-ending punctuation. For each word, we feed up to 48 words on either side into the Romanian BERT model `dumitrescustefan/bert-base-romanian-cased-v1`. The word vector is the mean of the target word’s subword hidden states. PCA reduces these embeddings to 128 dimensions; the cached run explained 65.76% of embedding variance.

5 Model and Training

The target is transformed as:

$$z_i = \sqrt{\max(y_i, 0)}, \quad \hat{y}_i = \max(\hat{z}_i, 0)^2. \quad (2)$$

This makes the regression problem less dominated by very large TRT values while keeping predictions non-negative after inversion.

Table 2: Final LightGBM configuration.

Parameter	Value
Trees	1000
Learning rate	0.022
Leaves	15
Minimum child samples	20
Subsample	1.0
Column sample by tree	0.7
L2 regularization	35
Target transform	square root
Fold scheme	5 folds $\times$ 2 seeds
Fold stratification threshold	90 ms

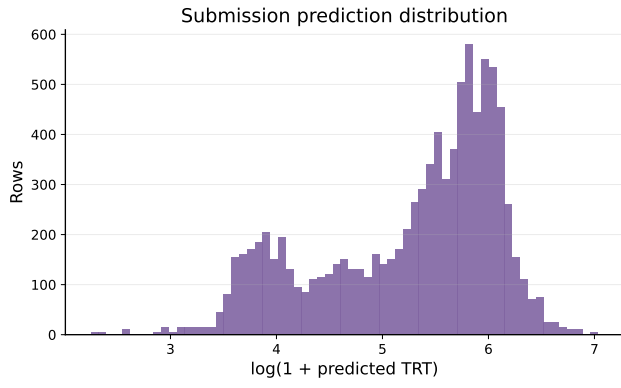
Categorical features are passed directly to LightGBM: genre, normalized word form, prefixes, suffixes, and shape variables. We run 5 stratified folds with seed 42 and 5 more with seed 123, then average all ten test predictions.

6 Results

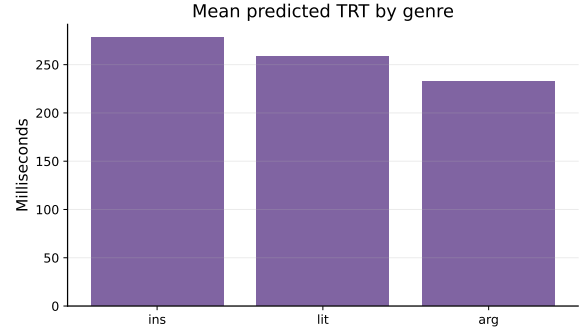
Table 3: Local validation and output diagnostics.

Metric	Value
Mean word-level CV score	87.19
Row-level OOF score	44.46
Row-level OOF RMSE	256.65 ms
OOF prediction mean	258.87 ms
OOF prediction std.	172.32 ms
Submission prediction mean	254.22 ms
Submission prediction median	249.01 ms
Submission prediction std.	162.16 ms
Submission prediction max	1,133.85 ms

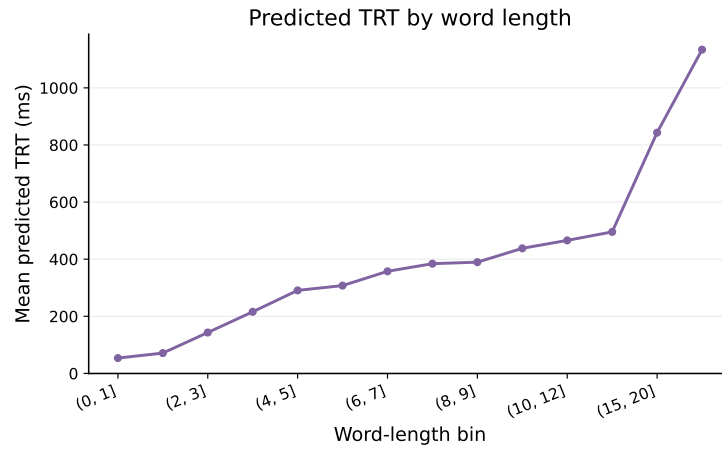
The word-level validation score is much higher than the row-level score because each word-level prediction is copied back to multiple participant rows. Participant-specific reading behavior remains noisy, but the model is intended to predict general word difficulty for unseen participant-word rows.



(a) Prediction distribution



(b) Genre means



(c) Length response

Figure 4: Submission diagnostics. Predictions are positive and preserve the expected relationship between word length and reading time.

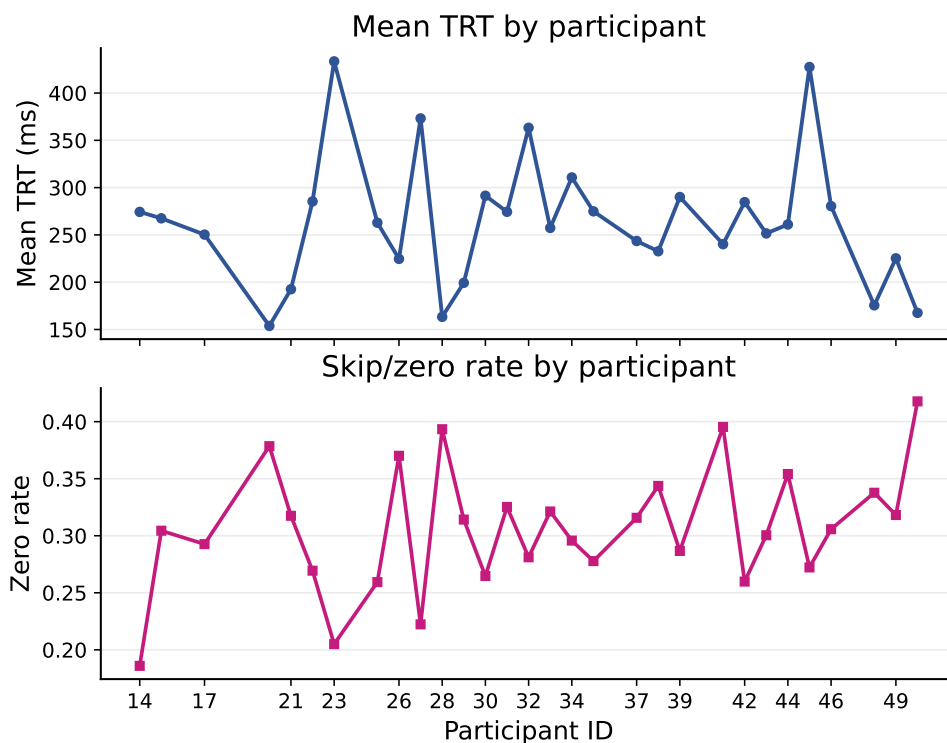


Figure 5: Participant-level variation in the training data. This motivates aggregating labels by word id.

7 What Worked

- Aggregating by `word_id` made the target substantially cleaner.
- Auxiliary fixation files were very valuable because they carried directly related behavioral signal.
- Romanian BERT embeddings helped add sentence context without fine-tuning.
- The square-root target transform stabilized LightGBM predictions.
- Repeated stratified folds reduced variance in the final submission.

8 Limitations

- The model does not explicitly model participant identity at inference time.
- PCA is fitted on train and test embeddings together. It uses no labels, but it is still transductive.
- The BERT encoder is frozen, so it is not specialized for eye-tracking signals.
- Auxiliary files are strong external data; comparisons should state clearly whether they are used.

9 Reproducibility

The final notebook is `main.ipynb`. Running it top-to-bottom writes `output.csv`. The first run may download Romanian BERT weights and build the embedding cache under `.cache`; later runs reuse the cached embeddings.

Main dependencies are `numpy`, `pandas`, `scipy`, `scikit-learn`, `torch`, `transformers`, `tqdm`, and `lightgbm`. Metric and scientific utilities rely on standard Python scientific packages [10, 11]. ChatGPT was used to help draft and organize this writeup and code presentation [9].

References

- [1] ana0101. eye-tracking: Romanian eye-tracking and trt modeling repository. <https://github.com/ana0101/eye-tracking>, 2025. Accessed 2026-04-26.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- [3] Stefan Dumitrescu, Andrei-Marius Avram, and Sampo Pyysalo. The birth of Romanian BERT. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4324–4328. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.387. URL <https://aclanthology.org/2020.findings-emnlp.387/>.
- [4] Anamaria Hodivoianu, Oleksandra Kuvshynova, Filip Popovici, Adrian Luca, and Sergiu Nisioi. Predicting total reading time using Romanian eye-tracking data. In *Proceedings of the First International Workshop on Gaze Data and Natural Language Processing*, pages 71–75, Varna, Bulgaria, 2025. INCOMA Ltd. URL <https://aclanthology.org/2025.gaze4nlp-1.9/>. PDF: <https://aclanthology.org/2025.gaze4nlp-1.9.pdf>.
- [5] Ian T. Jolliffe. *Principal Component Analysis*. Springer, 2 edition, 2002. doi: 10.1007/b98835.
- [6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree>.
- [7] MultipleEYE COST Action CA21131. Multipleye: Enabling multilingual eye-tracking data collection for human and machine language processing research. <https://multipleye.eu/>, 2026. Accessed 2026-04-26.
- [8] Nitro Association. Nitro NLP Hackathon. <https://nitro-ai.org/hackathon/>, 2026. 23-hour NLP competition, 25–26 April 2026. Accessed 2026-04-26.
- [9] OpenAI. ChatGPT. <https://chat.openai.com/>, 2026. Used for drafting, code organization, and writeup assistance; empirical claims are derived from the local data and cited sources.
- [10] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [11] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, Ilhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antonio H. Ribeiro, Fabian Pedregosa, and Paul van Mulbregt. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.